Full name: Deborah Onyebuchi Olisa Egonu
Student ID: 301915997
Student Email: d.egonu5997@miva.edu.ng
Course: MIT 8103 - Advanced Database Systems
Organisation Type: E-Commerce Platform
Case Study: NaijaMart (simulated Lagos-based e-commerce platform)
Github Repository: Egonu_Deborah_Onyebuchi_301915997

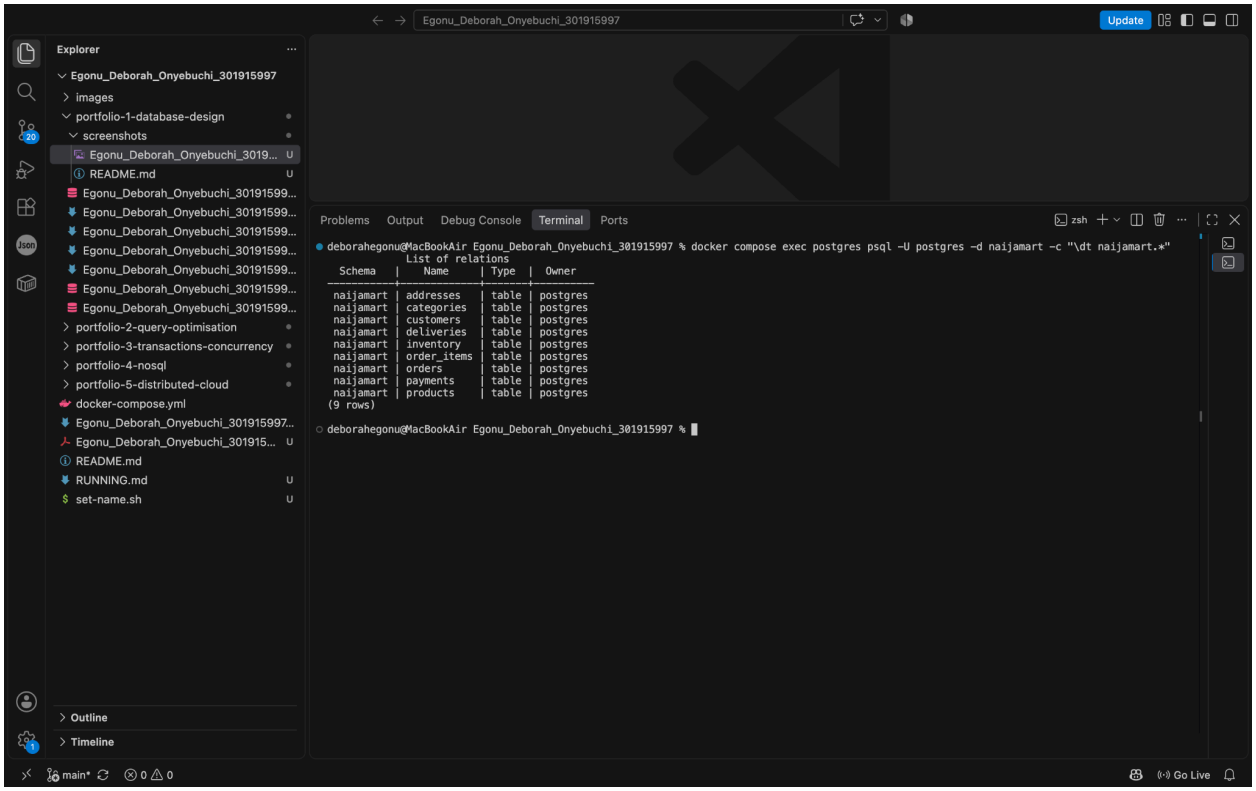
1. Case Study

Naijamart is a simulated Lagos-based online marketplace (E-commerce platform) where customers can browse products, place orders, pay and track delivery, while staff can manage stock and sales reports. The simulated database stores customers, addresses, categories, products, inventory, orders, order items, payments, and deliveries. It also supports placing orders, updating stock, payment collection, delivery tracking, and reporting revenue.

2. Portfolio 1: Database design and modelling

In portfolio 1, I designed the NaijaMart relational database using PostgreSQL in Docker Desktop. I created nine related tables to manage customers, addresses, products, inventory, orders, order items, payments, deliveries, and product categories.

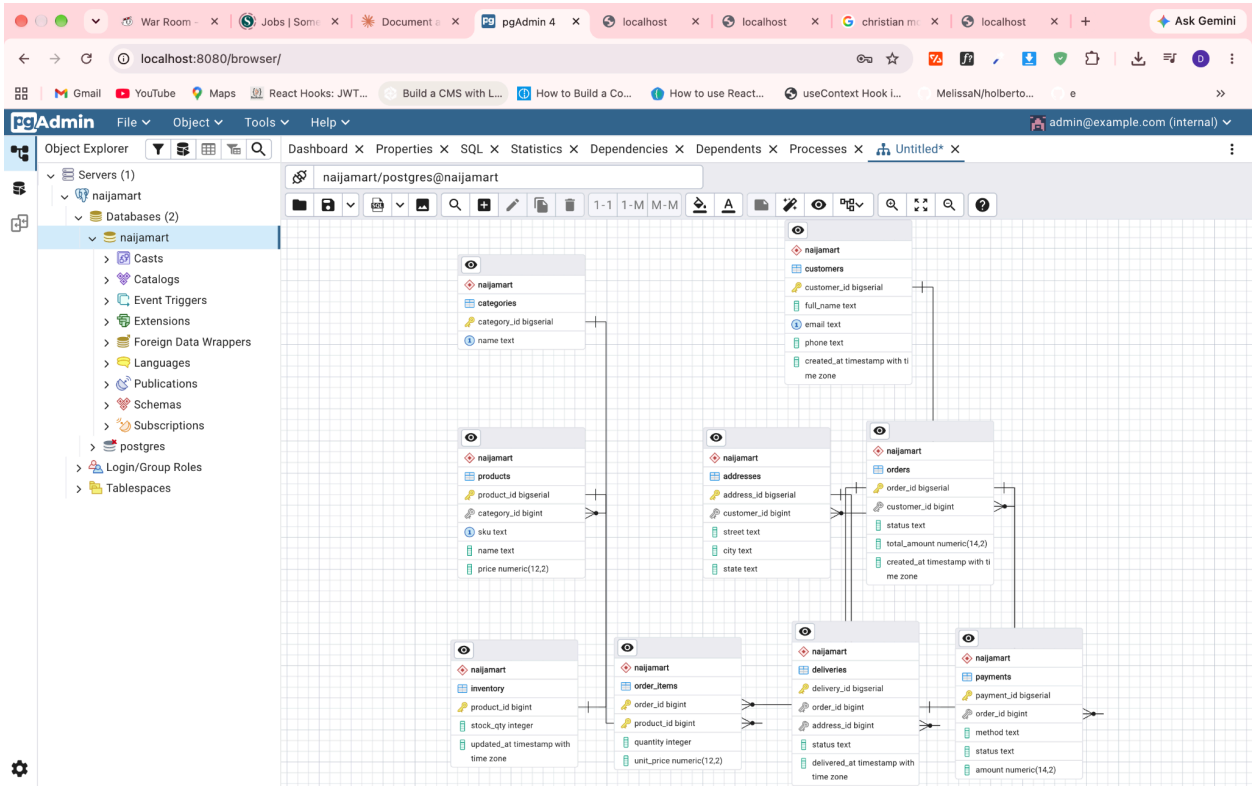
Figure 2.1: Screenshot of the nine tables created in PostgreSQL using Docker Compose



I used nine tables instead of one because different information belongs to different tables and separating the data accordingly makes the database easier to maintain and scale.

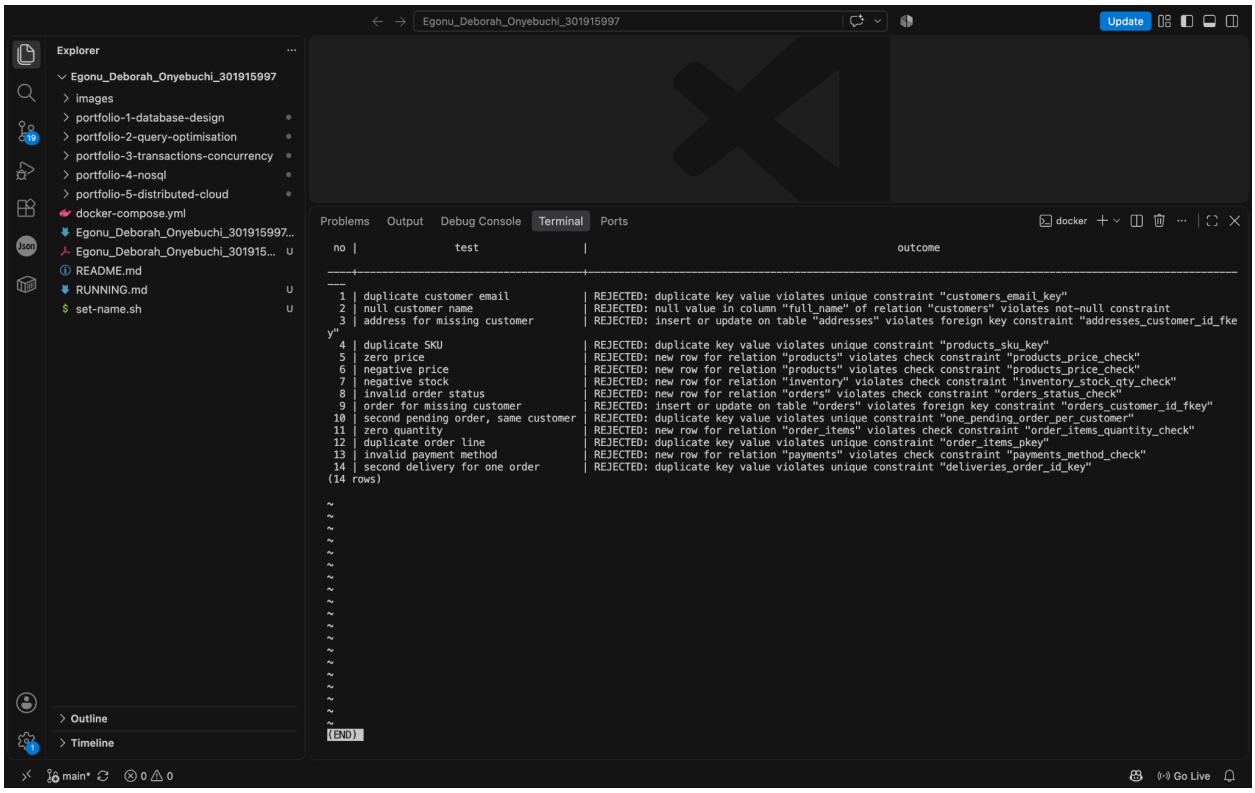
I established relationships between tables using primary and foreign keys demonstrated in the ER diagram created in PostgreSQL.

Figure 2.2: Screenshot of the Entity Relationship (ER) Diagram generated in PostgreSQL



I applied constraints such as NOT NULL (to prevent important fields from being left empty), UNIQUE (to prevent duplicate values), and CHECK (to prevent invalid values such as negative prices or stock quantities) to maintain data accuracy and integrity. I also used normalisation to reduce data duplication. Finally, I tested the constraints using the SQL scripts to confirm that invalid records were rejected.

Figure 2.3: Screenshot of the tested constraints using SQL scripts via Docker compose to confirm that invalid records were rejected.



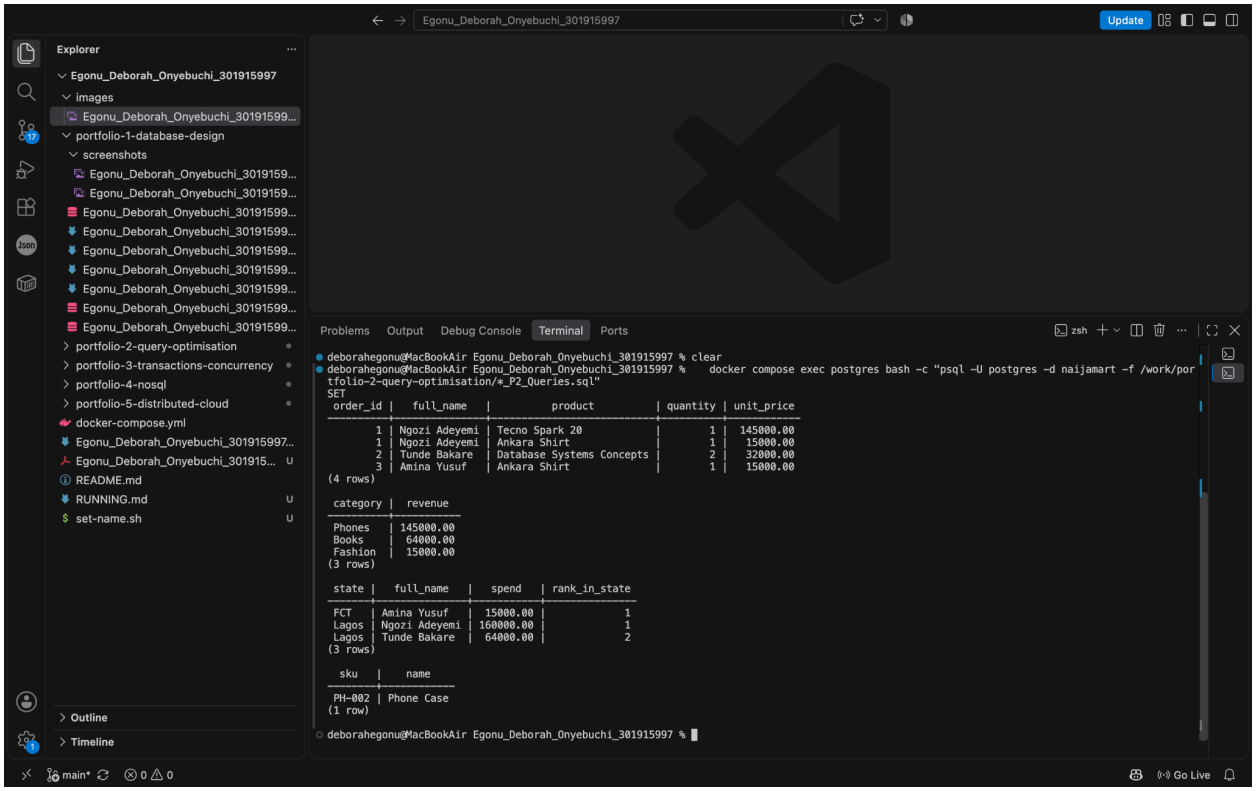
Files used in portfolio 1:

- P1_Schema.sql: Holds the SQL commands used to create the tables, keys, and constraints.
- P1_SampleData.sql: Holds the example customers, products, and transactions inserted into Naijamart database.
- P1_ConstraintTests.sql: Holds the tests performed to check that the Naijamart database rejects invalid data.

3. Portfolio 2: Query Processing and Optimisation

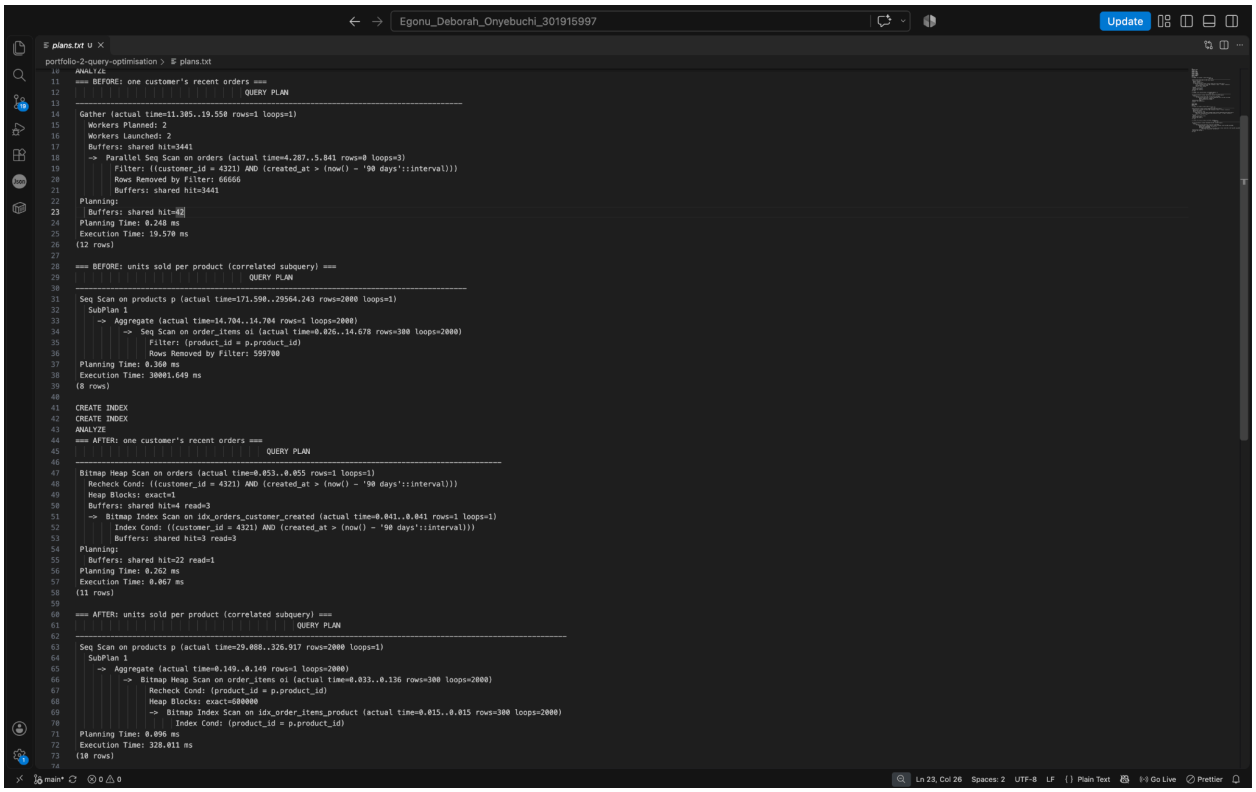
In portfolio 2, I wrote four SQL queries to retrieve information from the database. The queries included table joins, revenue per category, customer spend rankings, and identifying products without sales. These queries were executed using docker commands in my VS Code terminal displayed below.

Figure 3.1: Screenshot of the results of the SQL Queries run via docker compose.



I generated a large dataset of 200,000 synthetic orders to evaluate query performance. I created indexes on the orders and order_items tables to improve data retrieval speed. I then used EXPLAIN ANALYZE in PostgreSQL to examine the execution plans.

Figure 3.2: Screenshot of the before and after optimisation results using PostgreSQL EXPLAIN ANALYZE tool, exported the result to plans.txt



The recorded results showed that the recent-orders query improved from 19.570ms to 0.067ms which is a 99.66% increase in retrieval speed.

Files used in portfolio 2:

- P2_Queries.sql: Contains the four SQL queries I ran in my VS Code terminal.
- P2_Optimisation.sql: Contains the index creation and optimisation commands.
- plans.txt: Holds the query execution plans and performance comparisons results exported from PostgreSQL EXPLAIN ANALYZE.

4. Portfolio 3: Transactions and Concurrency

In portfolio 3, I attempted selecting a product that had a stock of 1 unit at the same time in two separate sessions to demonstrate the risk of overselling and the importance of transaction management and concurrency control. I first selected a product with a stock of 999 unit and updated it to 1 stock. I then used BEGIN (starts a transaction) , COMMIT (saves the transaction permanently) ,and ROLLBACK (cancels the transaction if anything goes wrong) to demonstrate how database operations can be saved or cancelled. In the two customers selecting one product at the same time simulation, I used SELECT FOR UPDATE to lock the inventory record during the transaction, ensuring that the second session waited and checked the updated stock before proceeding.

Figure 4.1: Screenshot evidence of COMMIT ROLLBACK in P3_Transactions.sql

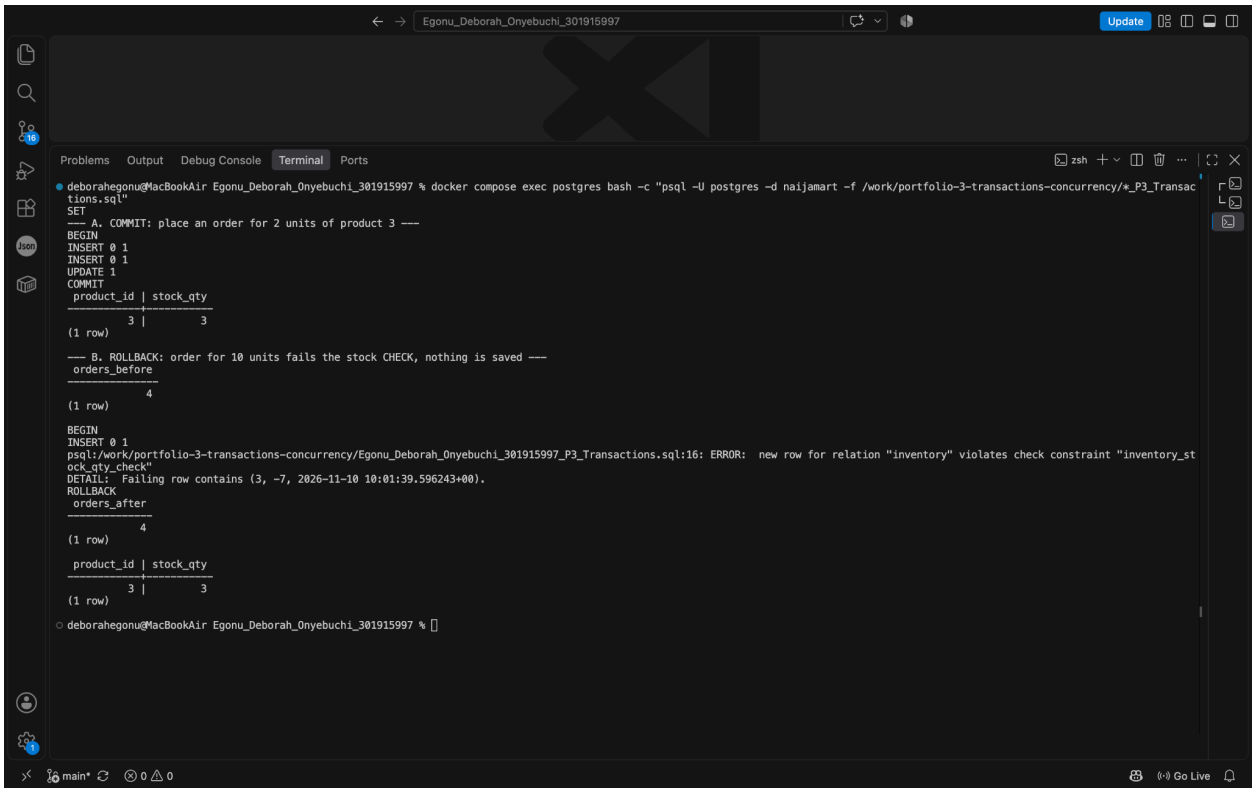


Figure 4.2: Screenshot evidence of concurrency lock wait that occurred while simulating two separate sessions to process the same product with a stock of 1 unit.

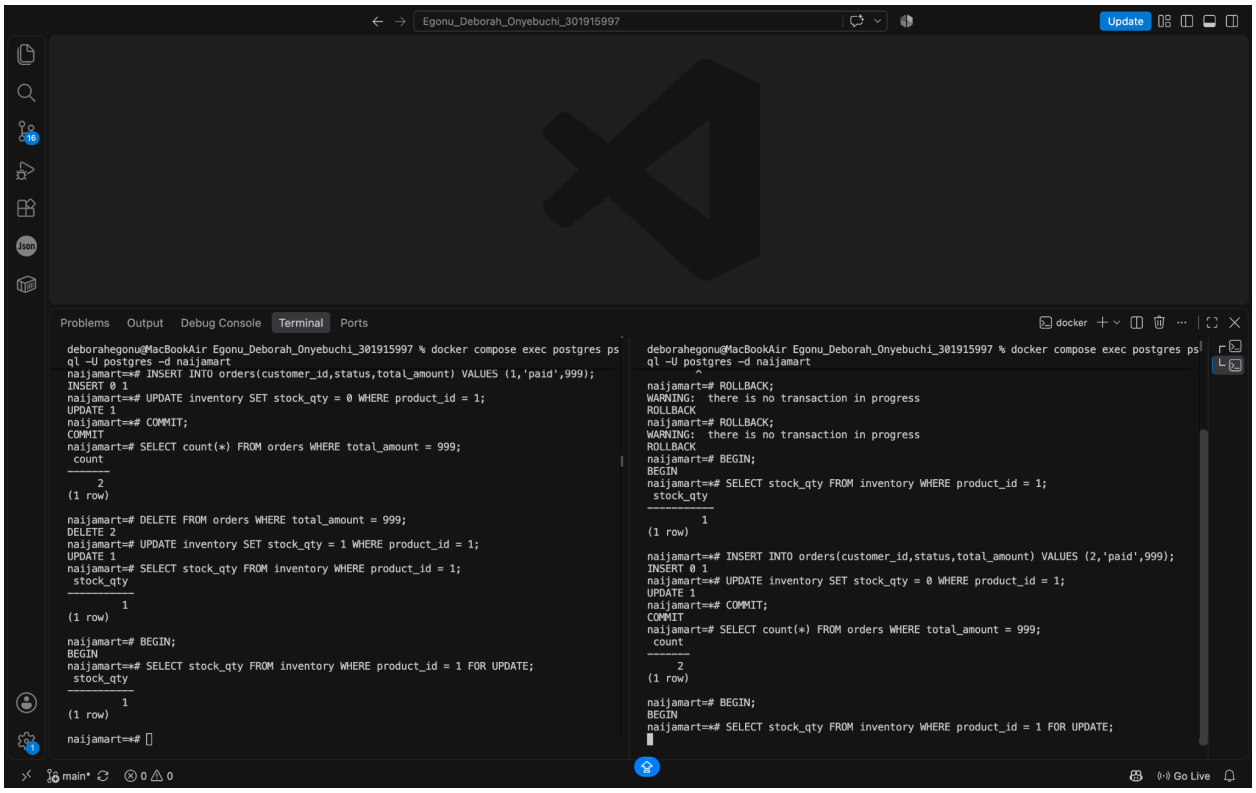
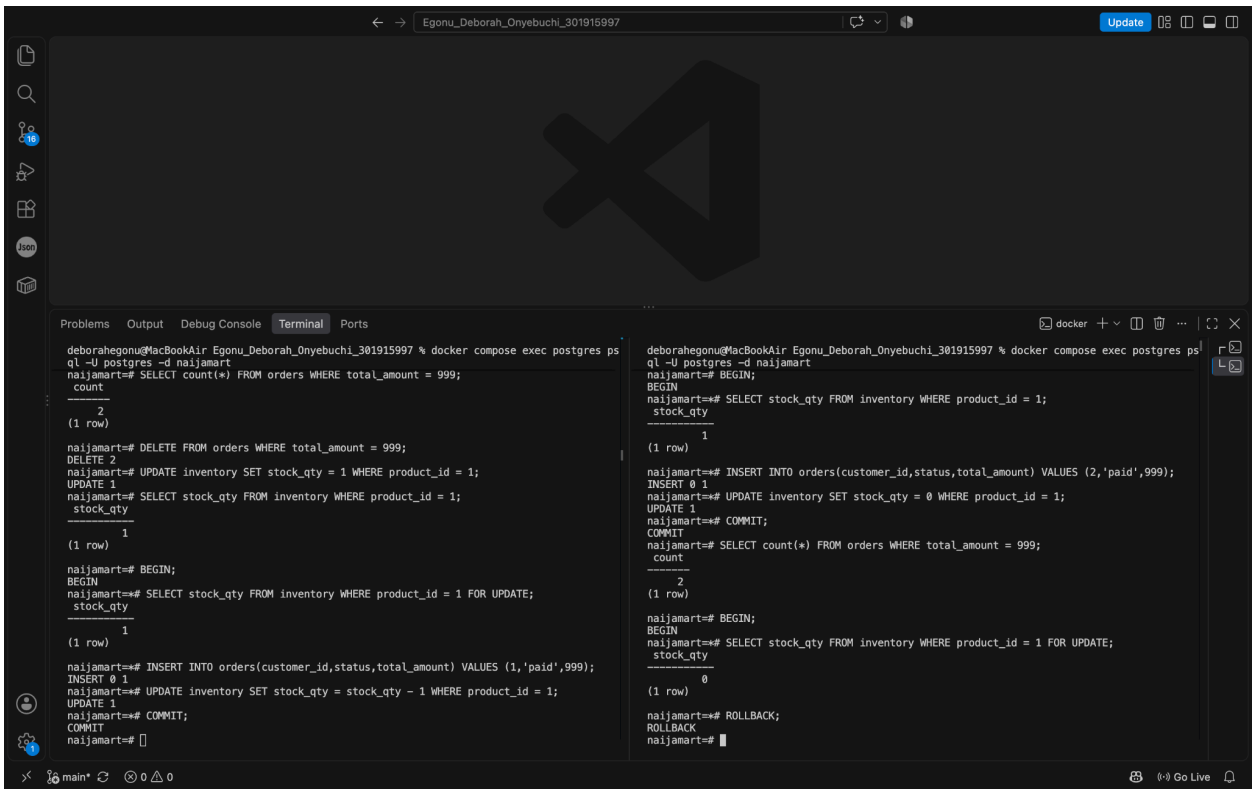


Figure 4.3: Screenshot evidence of the concurrency result.



Files used in portfolio 3:

- P3_Transactions.sql: Contains the transaction examples, including COMMIT and ROLLBACK.
- P3_SessionA.sql: Contains the first database session attempting to process an order.
- P3_SessionB.sql: Contains the second database session attempting to access the same product.

5. Portfolio 4: NoSQL and Advanced Data Models

In portfolio 4, I explored NoSQL databases such as MongoDB and Redis. I used MongoDB to model the product catalog because different products have different attributes. I implemented product documents, embedded reviews , schema validation, indexes, and aggregation queries.

Figure 5.1: Screenshot evidence of MongoDB product document and queries (1)

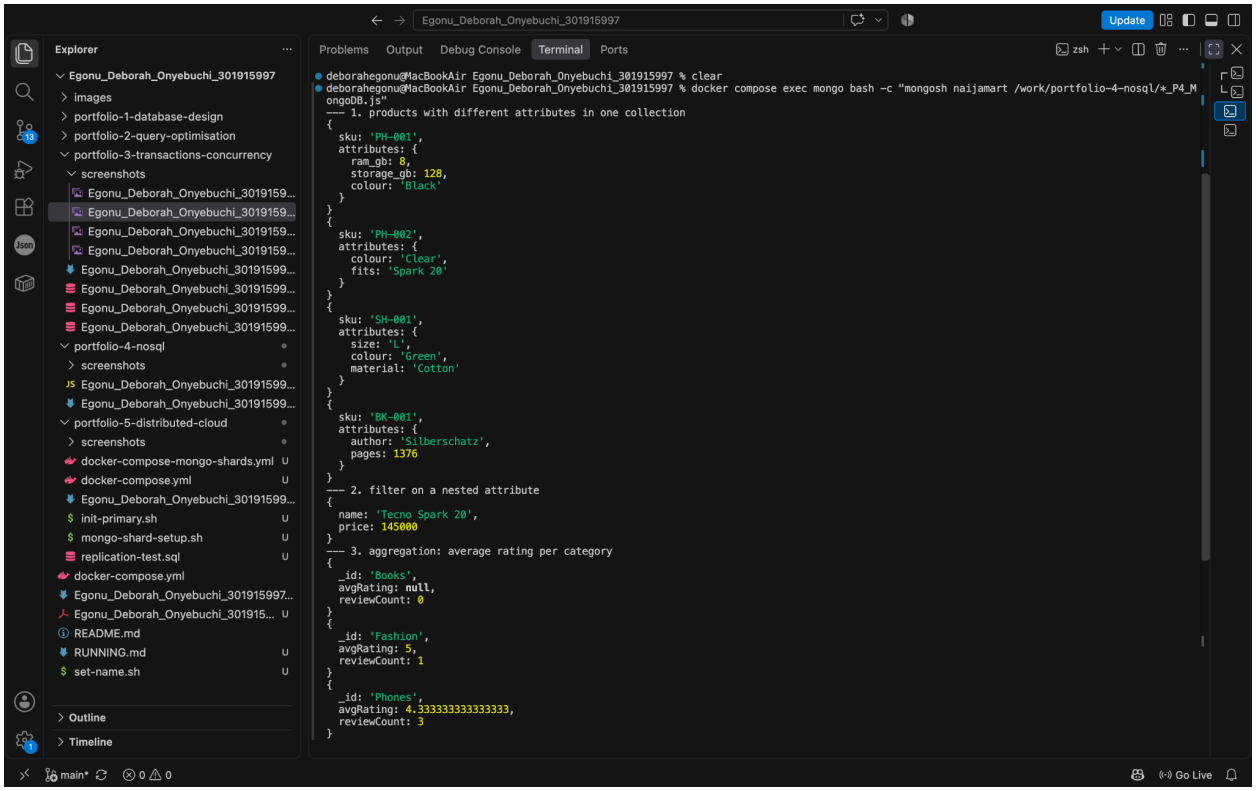
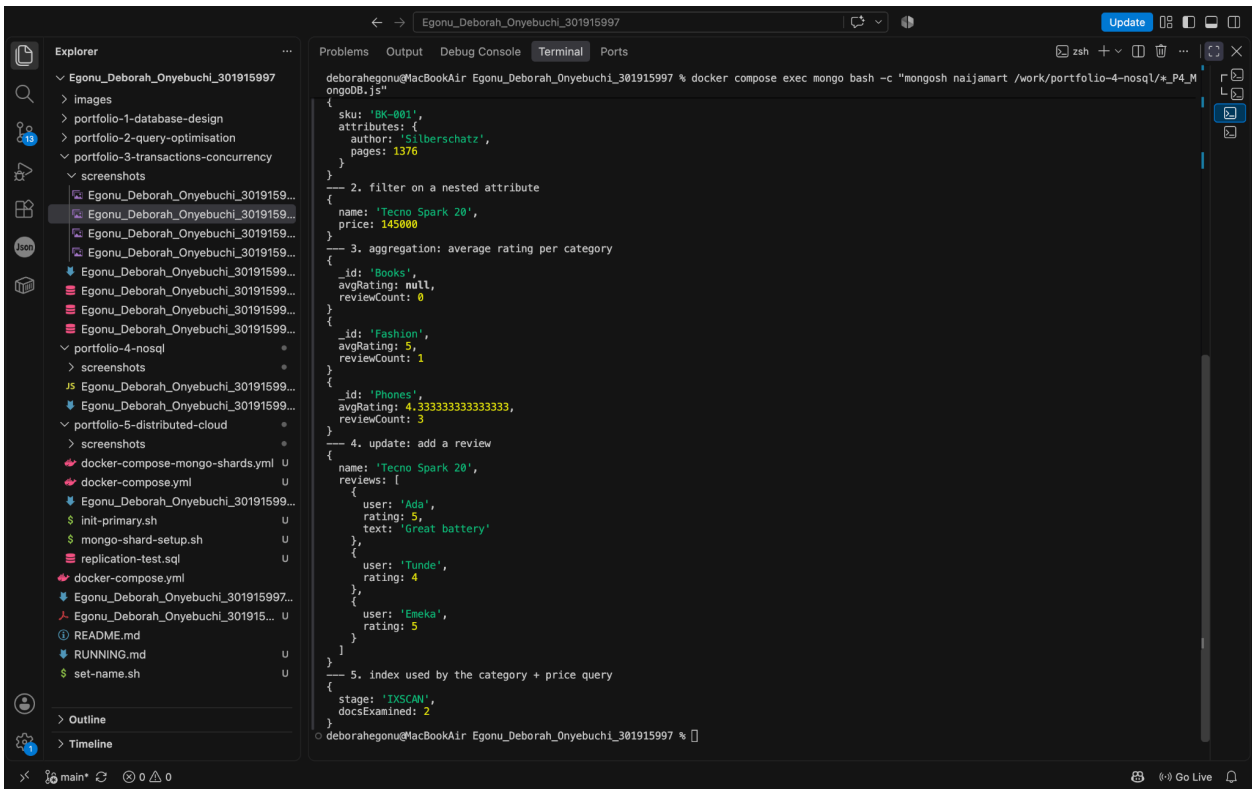
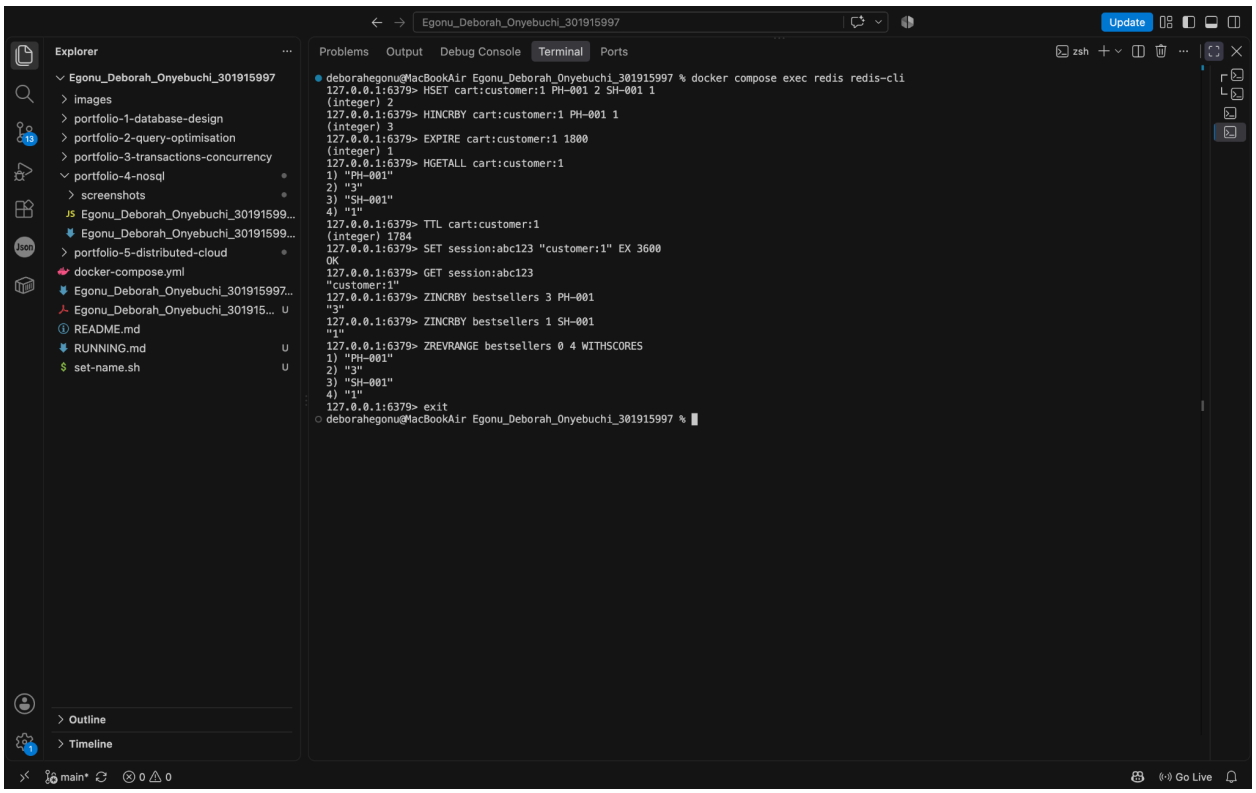


Figure 5.2: Screenshot evidence of MongoDB product document and queries (2)



I used Redis to demonstrate temporary scenarios such as shopping cart and session storage which included expiry settings and sorted sets for bestseller rankings.

Figure 5.3: Screenshot evidence of executed Redis commands



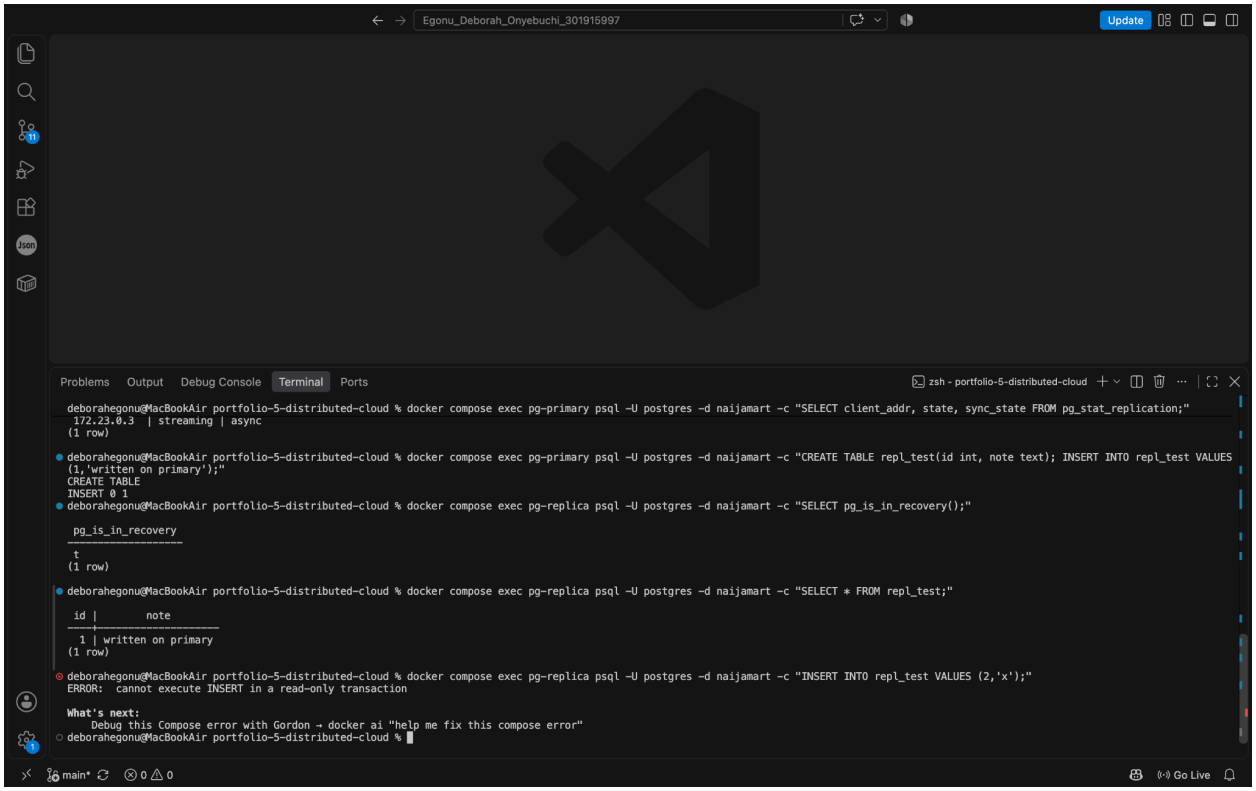
File used in portfolio 4:

- P4_MongoDB.js: Shows the MongoDB implementation, including document insertion, validation, indexes, queries, aggregation, and updates.

6. Portfolio 5: Distributed and Cloud Database Exercise

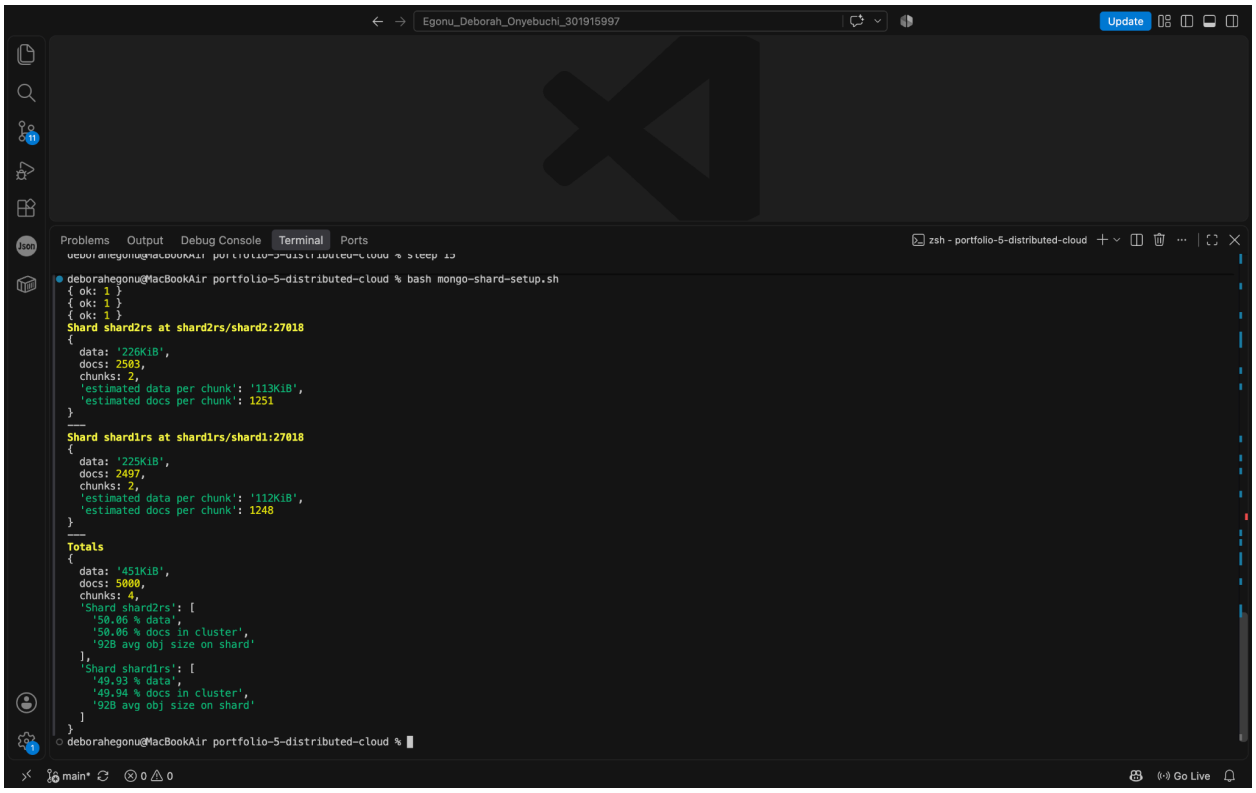
In portfolio 5, I demonstrated 2 distributed database concepts using PostgreSQL and MongoDB. In PostgreSQL, I configured PostgreSQL streaming replication (creating copies of data on other servers) to demonstrate how database changes can be copied from a primary database to a backup or standby.

Figure 6.1: Screenshot evidence of PostgreSQL streaming replication via Docker Compose



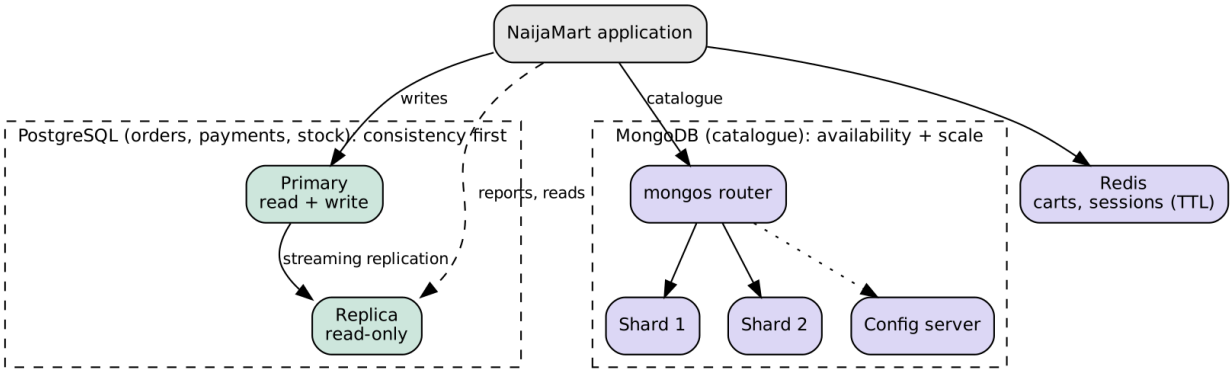
I also configured a MongoDB sharded-cluster environment (dividing data across servers) with a configuration server, two shards, and a mongos router. I used a hashed SKU shard key to demonstrate data distribution.

Figure 6.2: Screenshot evidence of MongoDB sharded-cluster environment



Finally, I proposed a cloud architecture using Amazon RDS for PostgreSQL and MongoDB Atlas for MondoDB which I generated using Claude AI.

Figure 6.3: Screenshot evidence of proposed cloud design generated with Claude AI



Files used in portfolio 5:

- Portfolio 5 docker-compose.yml : Contains the PostgreSQL replication environment.
- replication-test.sql: Contains the SQL used to test the replication setup.

- docker-compose-mongo-shards.yml: Defines the MongoDB sharded-cluster environment.
- init-primary.sh: Contains the primary database initialisation steps.
- mongo-shard-setup.sh: Contains the MongoDB sharding setup commands.

7. Individual Technical Reflection

I used Docker Desktop to test and complete this project instead of installing individual databases and applications to save my system resources. I am familiar and comfortable with PostgreSQL and MongoDB but using Redis was a learning experience. I ran into a few challenges while attempting the transactions and concurrency management in PostgreSQL gui. I was able to resolve the issue by moving to my VS code terminal and running the same commands to achieve the concurrency lock wait. I was able to retrace my steps to produce the desired results which enhanced my understanding of transactions and concurrency management. Finally, in portfolio 5, my MongoDB shard members are single-node replica sets because this is a local sharding demonstration, not a production-ready highly available cluster.

8. AI Declaration

Claude (Anthropic) AI was used to draft the case study, docker-compose.yml files, sample SQL and MongoDB scripts, and the generic technical report texts in each portfolio which I updated with the correct data.

ChatGPT (Open AI) was used to review the completed project files.

I ran every script myself, took screenshots of the checked outputs and plans, corrected errors, and wrote my own reflection, reports, and README.md.

9. References & Guides

PostgreSQL Documentation
MongoDB Documentation
Redis Documentation
Docker Compose Documentation